

NOM: MASSASSI

PRENOM: GANDOU PAUL

PROJET: NFE 204 Année 2014-2015

E-MAIL: massassipaul@yahoo.fr

RAPPORT PROJET NFE 204

Sujet: Étude sur le système NoSQL CouchDB

PLAN

Nous allons faire une étude détaillée sur le système NoSQL CouchDB à travers de ce support, l'étude sera basée sur la partie en mode locale et sur la partie en mode distribuée.

I- Partie

Il s'agira d'élucider quelques concepts du système CouchDB à partir d'un nœud.

I-1 - Nous présenterons CouchDB

Nous parlerons ici de façon globale du système CouchDB

I-2 - Comment installer CouchDB

I-3 - Quelques opérations usuelles

I-4 - Comment mettre en œuvre la réplication

I-5 - Importation de données

II- Partie

Il s'agira de montrer comment s'effectue la scalabilité et le partitionnement avec BigCouch sur un système multi-nœud

II - 1 - Présentation de BigCouch

II- 2 - Comment installer BigCouch

II- 3 - Mise en œuvre d'un système distribué avec BigCouch

I-1 - Présentation de CouchDB

CouchDB est une base de données documentaire au format JSON, il a été réalisé sous la licence Apache 2.0 et est basé sur le langage Erlang.

Architecture

L'architecture de CouchDB se compose d'un serveur unique, le moteur de stockage embarque son propre serveur HTTP, permettant aux clients d'effectuer des requêtes via l'API

Client -----Requête HTTP----->SERVEUR

CouchDB utilise le protocole HTTP pour la communication inter-nœud, pour accéder à une base ou pour accéder à un document contenu dans une collection en se servant des méthodes HTTP (POST, PUT, GET et DELETE). L'utilisation du protocole HTTP rend disponible les documents sur un navigateur web et à travers une connexion réseau.

Chaque document est caractérisé par un identifiant unique et se présente comme un ensemble d'éléments sous forme clé/valeur (format JSON), ce qui rend rapide et facile l'accès à une information.

Sur un document quelconque, on peut associer des pièces jointes (PDF et XML).

CouchDB est doté d'un:

- Back-office pour la gestion des bases de données

- Système de versionning documentaire, pour la gestion de différentes versions de chaque document, lorsqu'un nouveau document est créé, CouchDB conserve l'ancienne version ce mécanisme lui confère la possibilité de résoudre les problèmes de conflits de lecture et surtout d'écriture car on peut faire plusieurs traitements simultanément avec CouchDB.

- Mécanisme de Réplication

La réplication peut être bidirectionnelle et la synchronisation se fait de façon automatique sur une base à une autre.

Client1 -----requête HTTP ----->SERVEUR 1
|
Réplication bidirectionnelle
|
Client2-----requête HTTP ----->SERVEUR 2

- Possibilité de développer une application offline

- Framework MapReduce basé sur JavaScript permet de créer des vues, des agrégats, de faire du reporting sur un sous ensemble d'éléments et permet de disposer d'une vue comme système de requête principale

Cohérence de données

Du fait de la conception de la base, les données ne peuvent pas être distribuées sur plusieurs serveurs dans l'architecture cela permet à CouchDB d'être l'un des seuls SGDB NoSQL à pouvoir respecter les contraintes ACID.

Transaction

CouchDB n'implémente pas de mécanisme de transaction comme la plupart de NoSQL

Tolérance aux pannes

La réplication permet de rendre disponible les données sur plusieurs bases dans plusieurs nœuds

Scalabilité

CouchDB, propose une scalabilité concernant le traitement, pour obtenir une scalabilité de données, il faut utiliser BigCouch (voir Partie II) qui est un outil complémentaire de CouchDB pour distribuer les données et les traitements

I-2 - Comment installer CouchDB

Mon choix s'est porté sur le Système d' exploitation Ubuntu 12.04 pour ce projet.

Pour installer CouchDB sur Ubuntu 12.04 il faut:

- Mettre à jour le gestionnaire de paquet en tapant les commandes:
 - `sudo apt-get upgrade`
 - `sudo apt-get update`
- Lancer l'installation en tapant:
 - `sudo apt-get install couchdb`
- Ne pas oublier d'installer Curl pour la communication avec le serveur
 - `sudo apt-get install curl`

I-3 Quelques opérations usuelles

Une fois installé pour mettre le serveur en marche on tape:

- `sudo /etc/init.d/couchdb start`

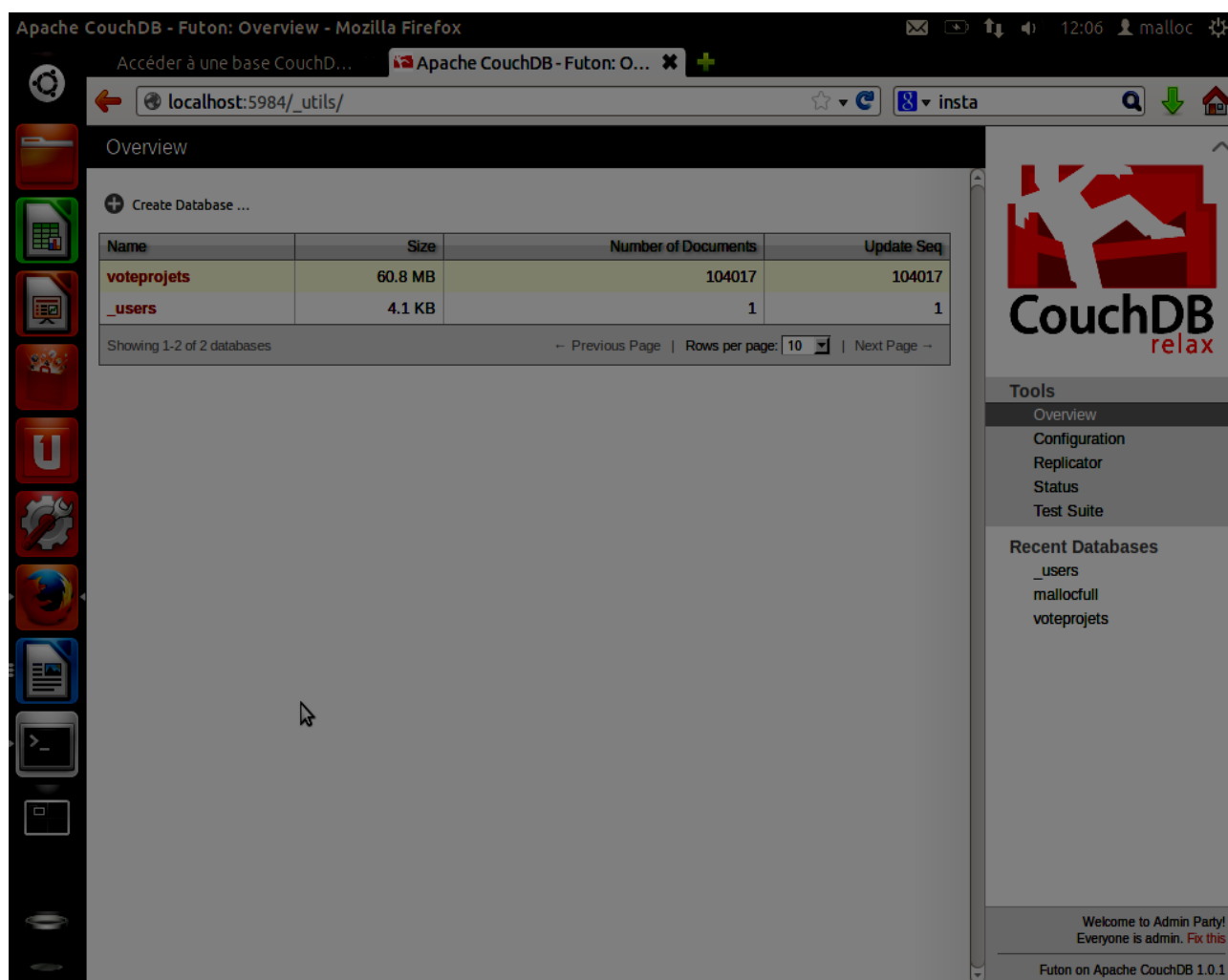
Pour l'arrêt on tape :

- `sudo /etc/init.d/couchdb stop`

Pour vérifier que le serveur est en marche on tape dans le terminal :

- **`curl http://localhost:5984`**, si le serveur est en marche il renvoie `{"couchdb":"Welcome","version":"1.0.1"}` comme retour, on peut accéder à l'interface d'administration (appelé futon) en allant sur lien **`http://localhost:5984/_utils/`**

Voici à quoi ressemble l'interface d'administration



Comme nous l'avons dit au départ CouchDB utilise le protocole HTTP, pour communiquer avec le serveur il nous faut utiliser les méthodes http (PUT, POST, DELETE et GET) pour communiquer avec lui

- GET: Pour récupérer une ressource
- PUT/POST: Pour créer/mettre/modifier/écrire une ressource
- DELETE: Pour supprimer une ressource

Méthode PUT/POST

Si nous voulons par exemple créer une base malloc on tape depuis le terminal:

- **curl -X PUT http://localhost:5984/malloc**, on a comme retour **{"ok":true}** la base malloc est créée il suffit de vérifier dans l'interface d'administration

Méthode GET

Si on veut récupérer la liste de toutes les bases de données on tape:

- **curl -X GET http://localhost:5984/_all_dbs**, notre serveur CouchDB contient les bases ["voteprojets", "_users", "malloc"] on voit bien que la base malloc a été créée.

Méthode DELETE

Si on veut supprimer une ressource ou une base on tape:

- **curl -X DELETE http://localhost:5984/malloc/**, on a **{"ok":true}** comme retour pour confirmer que la base malloc a été supprimée

on retape **curl -X GET http://localhost:5984/_all_dbs**, on obtient ["voteprojets", "_users"], on voit bien que la base malloc ne fait plus parti de la liste.

Les méthodes POST/PUT/DELETE/GET sont les méthodes de base pour communiquer avec un serveur CouchDB, même dans le cadre d'ajouter, de supprimer, de modifier, de récupérer un document dans une base ou un élément précis dans un document.

I-4 - Comment mettre en œuvre la réplication

On distingue sur CouchDB deux types de réplication: unidirectionnelle et bidirectionnelle, une réplication peut se faire également de façon continue pour l'illustration nous créons deux bases beta1 et beta2

```
beta1: curl -X PUT http://localhost:5984/beta1
beta2: curl -X PUT http://localhost:5984/beta2
```

Réplication unidirectionnelle

Ajoutons le document `Projet{ 'titre': projet }` dans beta1
`curl -X PUT http://localhost:5984/beta1/Projet -d '{"titre":"projet"}'` on a comme retour
`{"ok":true,"id":"Projet","rev":"1-24acb747c50d66dc0ecfdec623740d8b"}`, le document s'est inséré dans beta1, maintenant vérifions si le document existe dans beta2 `curl -X GET http://localhost:5984/beta2/Projet` renvoie `{"error":"not_found","reason":"missing"}` ce qui confirme que le document n'est pas dans beta2. Appliquons maintenant la réplication unidirectionnelle de la base beta1 vers beta2, on tape la commande :

```
curl -H "Content-type:application/json" -X POST
http://localhost:5984/_replicate -d
'{"source":"http://localhost:5984/beta1","target":"http://localhost:5984/beta2"}'
```

Puis vérifions si le document `Projet` est bien dans beta2

```
curl -X GET http://localhost:5984/beta2/Projet
```

```
{"_id":"Projet","_rev":"1-24acb747c50d66dc0ecfdec623740d8b","titre":"projet"}, c'est le cas
```

conclusion nous avons fait une duplication du document `Projet` vers beta2

Réplication Bidirectionnelle

Il suffit de taper une fois

```
curl -H "Content-type:application/json" -X POST
http://localhost:5984/_replicate -d
'{"source":"http://localhost:5984/beta1","target":"http://localhost:5984/beta2"}'
```

Puis inverser la base source et la base target


```
curl -H "Content-type:application/json" -X POST
http://localhost:5984/_replicate -d
'{"source":"http://localhost:5984/beta2","target":"http://localhost:5984/beta
1" }'
```

Réplication continue

Nous souhaitons faire une réplication continue c'est à dire à chaque insertion d'élément dans beta1, la duplication se fait automatiquement dans beta2 on tape dans le terminal la commande:

```
curl -H "Content-type:application/json" -X POST
http://localhost:5984/_replicate -d
'{"source":"http://localhost:5984/beta1","target":"http://localhost:5984/bet2
", "continuous":true}'
```

Testons :

insérons dans beta1 le document Theme{theme : etude CouchDB} tels que:

```
curl -X PUT http://localhost:5984/beta1/Theme -d '{"Theme":"etude
CouchDB"}'
```

vérifions si la duplication s'est faite instantanément dans beta2 en tapant curl -X GET http://localhost:5984/beta2/Theme on a :

```
{"_id":"Theme","_rev":"1-
26db4df60c390fc266394b334136b6d6","Theme":"etude CouchDB"} ,
conclusion la réplication est fait automatiquement sans avoir à retaper la
commande de réplication.
```

Afin de confirmer qu'elle se fait de façon automatique, considérons le document Auteur {auteur : paul} tels que :

```
curl -X PUT http://localhost:5984/beta1/Auteur -d '{"auteur":"paul"}' que
l'on insère dans beta1
```

Vérifions si la duplication s'est faite instantanément dans beta2 en tapant curl -X GET http://localhost:5984/beta2/ Auteur, on a bien:

```
{"_id":"Auteur","_rev":"1-
997f505c74c66340b7333d210e5fc519","auteur":"paul"}, ce qui confirme
bien qu'elle se fait de manière continue.
```

Nous avons illustré ici comment se fait la réplication continue unidirectionnelle pour une Bidirectionnelle il suffit d'inverser la source et le target. Ce type réplication est très indispensable

I-5 Importation de données

Dans le cadre de ce projet, pour l'importation, je dispose d'une source donnée que j'ai repérée sur le site

<http://opendata.paris.fr/explore/dataset/statistiques-des-votes-en-ligne-du-budget-participatif-2014/?tab=table>

Il s'agit d'une collecte de données au format JSON sur les résultats d'un vote qui avait lieu entre septembre et octobre 2014. Les participants ont voté chacun au plus cinq propositions de projets sur les budgets participatifs dans chaque arrondissement de paris, la volumétrie est de données de l'ordre de 104017 éléments et de taille des données 60.8 Mégaoctets.

Pour importer dans la base voteprojets j'ai utilisé la commande:

```
- curl -H "Content-type:application/json"-d @voteProjets.json -X POST  
http://127.0.0.1:5984/voteprojets/_bulk_docs , ce qui donne:
```

Apache CouchDB - Futon: Browse Database - Mozilla Firefox

localhost:5984/_utils/database.html?voteprojets

Overview voteprojets

+ New Document Security... Jump to: Document ID View: All documents Stale views

Compact & Cleanup... Delete Database...

Key	Value
"dd4500ae9fcb6b2156516158c0fffcf8" ID: dd4500ae9fcb6b2156516158c0fffcf8	{rev: "1-6f3d82061dd914d4d181aaf8daa353bd"}
"dd4500ae9fcb6b2156516158c0fffeb6" ID: dd4500ae9fcb6b2156516158c0fffeb6	{rev: "1-8be53c6bc4d1e247796573b1a6608f04"}
"dd4500ae9fcb6b2156516158c0fffee20" ID: dd4500ae9fcb6b2156516158c0fffee20	{rev: "1-58b6bde521bc91765b5583d72cb8c339"}
"dd4500ae9fcb6b2156516158c0fffe60b" ID: dd4500ae9fcb6b2156516158c0fffe60b	{rev: "1-72d0a77bb6fd873c3236ac178dd50cf"}
"dd4500ae9fcb6b2156516158c0ffdd1e" ID: dd4500ae9fcb6b2156516158c0ffdd1e	{rev: "1-a6ed3ed01adf46278d207d08907f3529"}
"dd4500ae9fcb6b2156516158c0fffd46f" ID: dd4500ae9fcb6b2156516158c0fffd46f	{rev: "1-8d145ec57054af95c4e310ed4067214e"}
"dd4500ae9fcb6b2156516158c0ffcc0b" ID: dd4500ae9fcb6b2156516158c0ffcc0b	{rev: "1-1adb00a148c7f70e4b4f70b3dc2607d8"}
"dd4500ae9fcb6b2156516158c0ffcd5" ID: dd4500ae9fcb6b2156516158c0ffcd5	{rev: "1-49c6f9e3d41c14b4828a46e91ad0292d"}
"dd4500ae9fcb6b2156516158c0ffb80a" ID: dd4500ae9fcb6b2156516158c0ffb80a	{rev: "1-5c3af38839f61e48b1079a6d6ca67973"}
"dd4500ae9fcb6b2156516158c0ffb2be" ID: dd4500ae9fcb6b2156516158c0ffb2be	{rev: "1-73fc3af8bb5bf1ead0d43044481cf44e"}
"dd4500ae9fcb6b2156516158c0ffaddc" ID: dd4500ae9fcb6b2156516158c0ffaddc	{rev: "1-3c0070a36a61ecbd27a84dfe99a5c0"}
"dd4500ae9fcb6b2156516158c0ffa337" ID: dd4500ae9fcb6b2156516158c0ffa337	{rev: "1-45bb90928c196e32a79be4da9c2eb31f"}
"dd4500ae9fcb6b2156516158c0ff96a7" ID: dd4500ae9fcb6b2156516158c0ff96a7	{rev: "1-d8ed6e35d603739aff630926758e27b6"}
"dd4500ae9fcb6b2156516158c0ff8816" ID: dd4500ae9fcb6b2156516158c0ff8816	{rev: "1-b6143276d9da168acd49c0ec7e26fe48"}
"dd4500ae9fcb6b2156516158c0ff82eb" ID: dd4500ae9fcb6b2156516158c0ff82eb	{rev: "1-548250d1d5e0fce3f1278f755f2d12e6"}

CouchDB relax

Tools

- Overview
- Configuration
- Replicator
- Status
- Test Suite

Recent Databases

- _users
- mallofull
- voteprojets

Welcome to Admin Party!
Everyone is admin. [Fix this](#)

Futon on Apache CouchDB 1.0.1

II - 1 - Présentation de BigCouch

Nous avons mise en œuvre la réplication sur la partie I, mais cette façon de faire à un inconvénient, lorsqu'on est confronté à des problèmes de données massives, lorsque la capacité de stockage d'un disque dans un nœud est limitée, c'est là qu'intervient BigCouch.

BigCouch est un produit créé par la société Cloudant sous licence apache 2.0, il s'intègre dans l'environnement CouchDB comme une brique additionnelle et se comporte comme un proxy qui répartit les requêtes vers les nœuds afin de garantir l'équilibrage de charges, il apporte de la scalabilité et permet de réaliser une architecture distribuée avec CouchDB qui dès le début n'est pas conçu pour ce type d'architecture. Il est comme CouchDB écrit en langage Erlang, utilise le protocole HTTP, peut contenir le type de support Blob, la recherche d'une information se fait par clé/valeur, dispose du Framework MapReduce pour créer des vues, le partitionnement se fait par hachage, il est hautement disponible, tolérant à la panne, utilise le nombre de quorum en cas de conflit.

BigCouch peut être utilisé pour créer des nœuds CouchDB, pour réaliser la distribution et la répartition de données BigCouch dispose de quatre paramètres (Q, N, W, R) que nous pouvons configurer selon nos besoins pour la réalisation d'une telle architecture.

-Q: Nombre de partition pour la division des documents d'une base

-N: Nombre de réplica de chaque document

-W: Nombre d'écriture quorum ($W < N$)

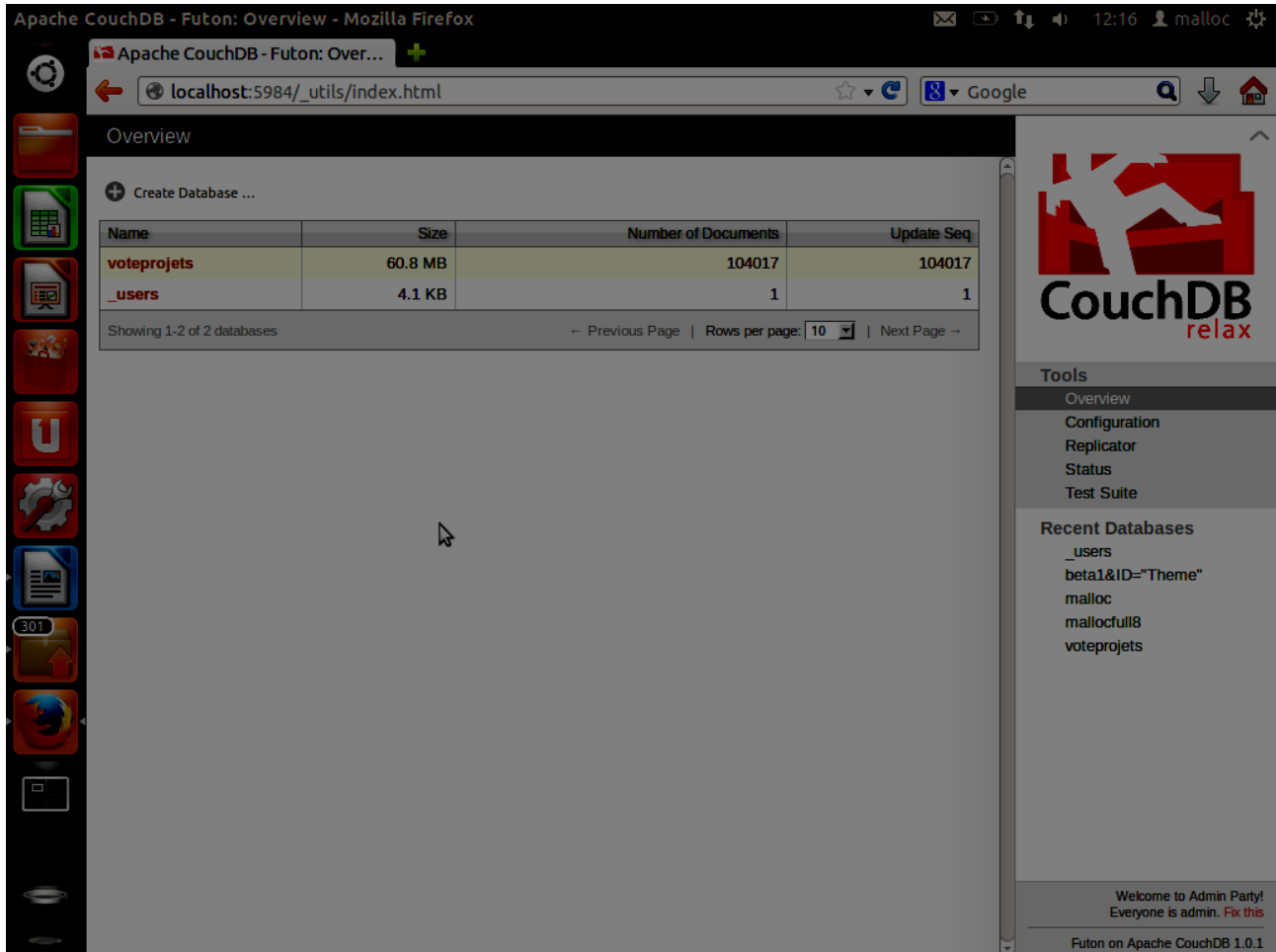
-R: Nombre de lecture quorum ($R < N$).

II-2 Comment installer BigCouch

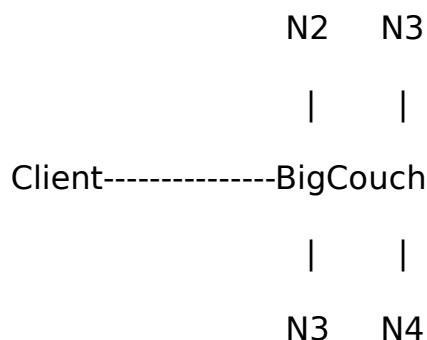
- Tout d'abord exécuter les commandes
- `sudo apt-get upgrade`
- `sudo apt-get update`
- Puis
- `sudo apt-get install git`
- `sudo apt-get update`
- Télécharger BigCouch via un gestionnaire de dépôt
 - `git clone https://github.com/cloudant/bigcouch.git`
- Puis faites dans l'ordre
 - `cd bigcouch`
 - `. /configure`
 - `make`
 - `make install`
 - `cp -r /home/votre_repertoire_courant/bigcouch /etc/sv`
 - `ln -s /etc/sv/bigcouch /etc/service/bigcouch`
- lancer BigCouch avec la commande
 - `sv start bigcouch`

II-3 Mise en œuvre d'un système distribué avec BigCouch

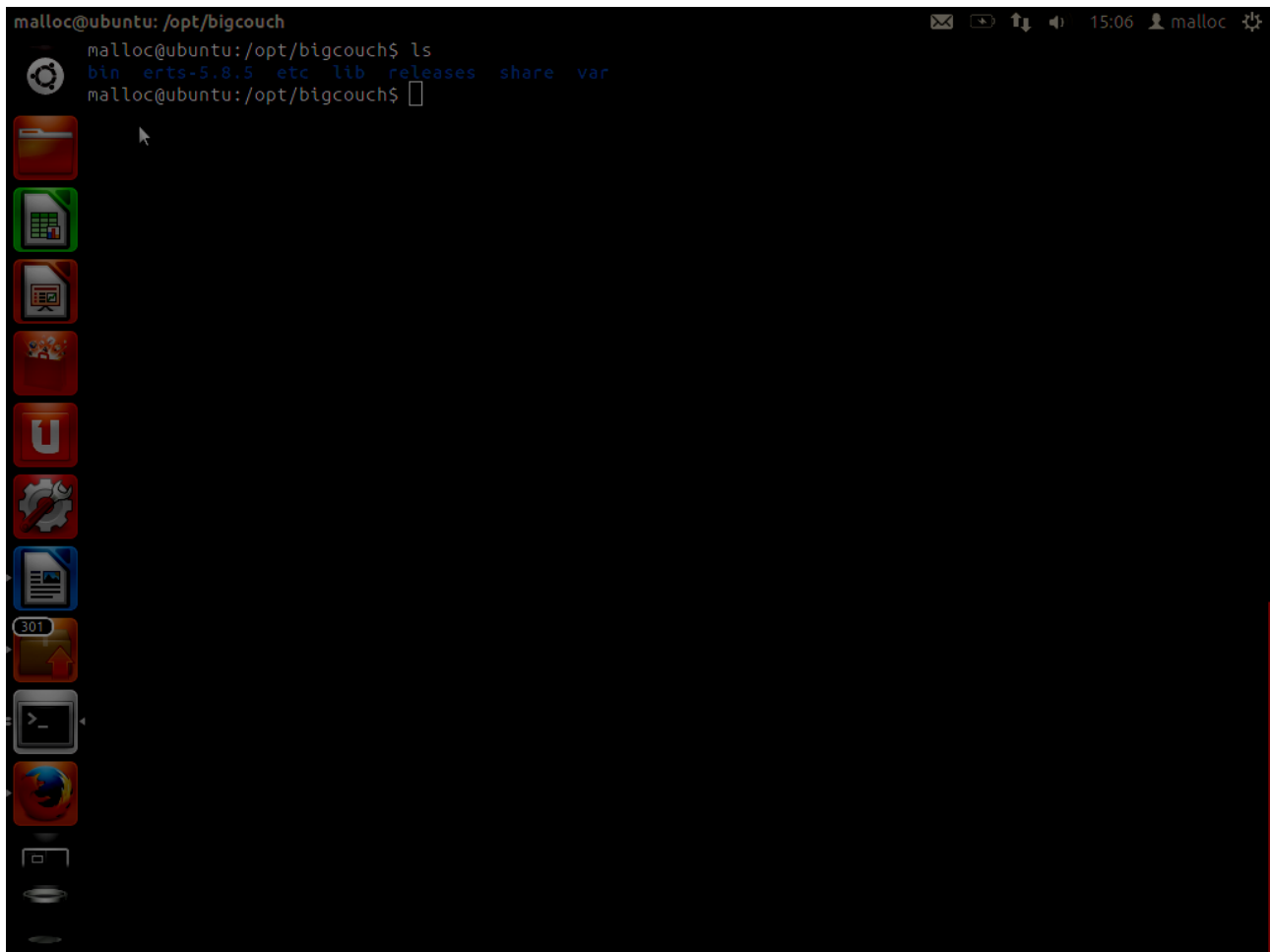
Dans cette section nous allons voir comment configurer BigCouch pour mettre en place un système scalable au niveau des données. Dans la partie I, nous avons importé dans la base voteprojets un fichier JSON de tailles 60,8 Mégaoctets, composé de 104017 éléments.



Cette façon de faire nous l'avions déjà dit n'est pas bonne dans le cas des données massives. Nous souhaitons repartir les 104017 éléments dans les fragments (partition), que BigCouch va repartir dans les nœuds. Nous allons donc créer quatre nœuds, chaque nœud pouvant contenir quatre partitions



Pour créer les nœuds N1, N2, N3, N4, créer les répertoires n1, n2, n3 et n4 dans /opt/bigcouch (ce chemin est créé automatiquement pendant l'installation bigcouch)

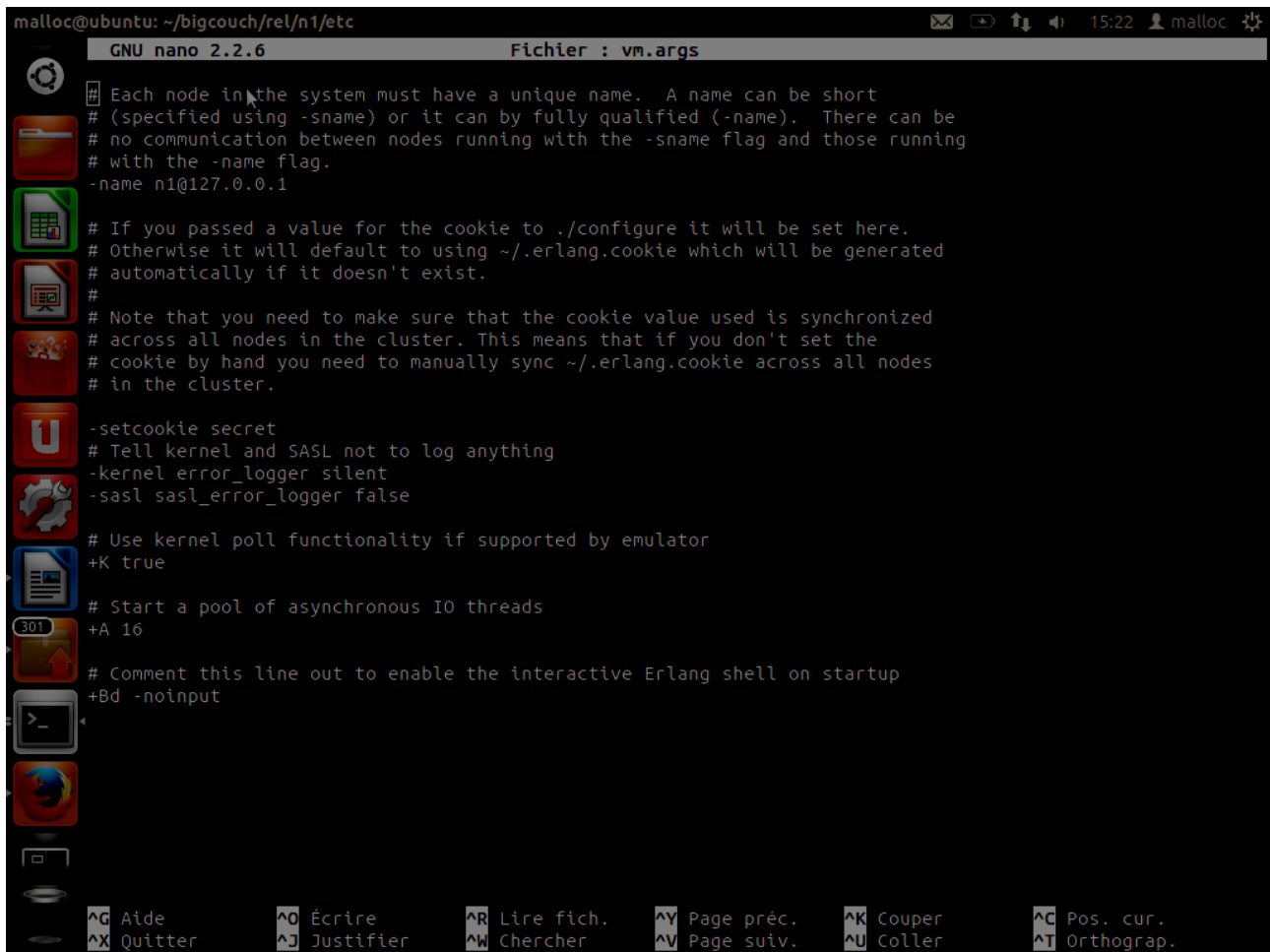
A terminal window on an Ubuntu system. The prompt is 'malloc@ubuntu: /opt/bigcouch'. The user has entered 'ls' and the output is 'bin erts-5.8.5 etc lib releases share var'. The terminal has a dark background with light blue text. On the left side of the terminal window, there is a vertical dock with several application icons: a folder, a spreadsheet, a presentation, a file manager, a terminal, a web browser, and others. The top of the terminal window shows system status icons like network, volume, and battery, along with the time '15:06' and the username 'malloc'.

Copier les répertoires (bin, erts, etc, lib, releases, share, var) dans les répertoires n1, n2, n3 et n4. Puis accéder à chaque répertoire « etc » de chaque répertoire n1, n2, n3 et n4, modifier le fichier vm.args en lui attribuant le nom du nœud au niveau du flag -name noeud@nomserveur et en attribuant un mot de passe au flag -setcookie votremotdepas le mot de passe doit être le même pour toutes les modifications.

Par exemple voici le fichier vm.args pour le nœud n1

-name « n1@127.0.0.1 »

-setcookie « secret » (pareil tous les autres)



```
mallocc@ubuntu: ~/bigcouch/rel/n1/etc
GNU nano 2.2.6 Fichier : vm.args
# Each node in the system must have a unique name. A name can be short
# (specified using -sname) or it can be fully qualified (-name). There can be
# no communication between nodes running with the -sname flag and those running
# with the -name flag.
-name n1@127.0.0.1

# If you passed a value for the cookie to ./configure it will be set here.
# Otherwise it will default to using ~/.erlang.cookie which will be generated
# automatically if it doesn't exist.
#
# Note that you need to make sure that the cookie value used is synchronized
# across all nodes in the cluster. This means that if you don't set the
# cookie by hand you need to manually sync ~/.erlang.cookie across all nodes
# in the cluster.

-setcookie secret
# Tell kernel and SASL not to log anything
-kernel error_logger silent
-sasl sasl_error_logger false

# Use kernel poll functionality if supported by emulator
+K true

# Start a pool of asynchronous IO threads
+A 16

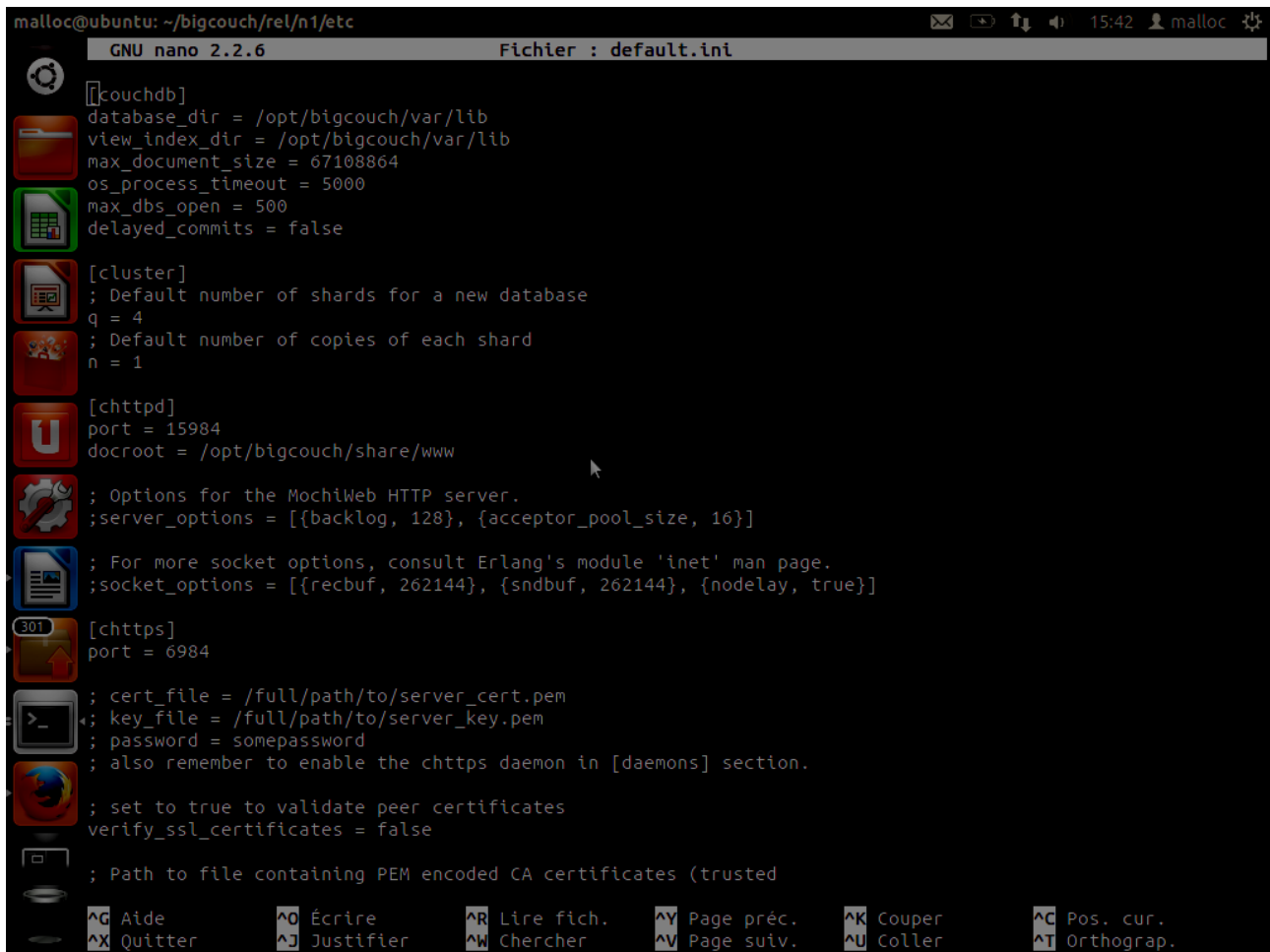
# Comment this line out to enable the interactive Erlang shell on startup
+Bd -noinput

^G Aide      ^O Écrire    ^R Lire fich.^Y Page préc.^K Couper
^X Quitter   ^J Justifier ^W Chercher  ^V Page suiv.^U Coller
^_           ^I           ^T Orthograp.
```

Toujours dans le répertoire « etc » pour tous les répertoires (n1, n2, n3, n4) configurer le fichier default.ini, c'est dans ce fichier qu'on va:

- Déterminer le nombre de partition q, par défaut il est égal 8 mais nous avons décidé d'attribuer q= 4
- Attribuer le nombre le nombre de réplica n pour chaque partition, nous avons choisi n=1
- Modifier les paramètres R, et W, mais nous avons décidé de les garder
- Attribuer les ports d'entrée et de sortie à chaque nœud (port unique)

Voici à quoi ressemble le fichier default.ini, pour le nœud n1, nous avons attribué les valeurs port d'entrée 15984 et port de sortie 15986 pour le nœud n1@127.0.0.1



The screenshot shows a terminal window with the nano text editor open, editing the file /opt/bigcouch/rel/n1/etc/default.ini. The terminal title bar indicates the user is 'mallocc' on an 'ubuntu' machine at '15:42'. The nano editor's status bar shows 'GNU nano 2.2.6' and 'Fichier : default.ini'. The file content is as follows:

```
[couchdb]
database_dir = /opt/bigcouch/var/lib
view_index_dir = /opt/bigcouch/var/lib
max_document_size = 67108864
os_process_timeout = 5000
max_dbs_open = 500
delayed_commits = false

[cluster]
; Default number of shards for a new database
q = 4
; Default number of copies of each shard
n = 1

[chttpd]
port = 15984
docroot = /opt/bigcouch/share/www

; Options for the MochiWeb HTTP server.
;server_options = [{backlog, 128}, {acceptor_pool_size, 16}]

; For more socket options, consult Erlang's module 'inet' man page.
;socket_options = [{recbuf, 262144}, {sndbuf, 262144}, {nodelay, true}]

[https]
port = 6984

; cert_file = /full/path/to/server_cert.pem
; key_file = /full/path/to/server_key.pem
; password = somepassword
; also remember to enable the https daemon in [daemons] section.

; set to true to validate peer certificates
verify_ssl_certificates = false

; Path to file containing PEM encoded CA certificates (trusted)
```

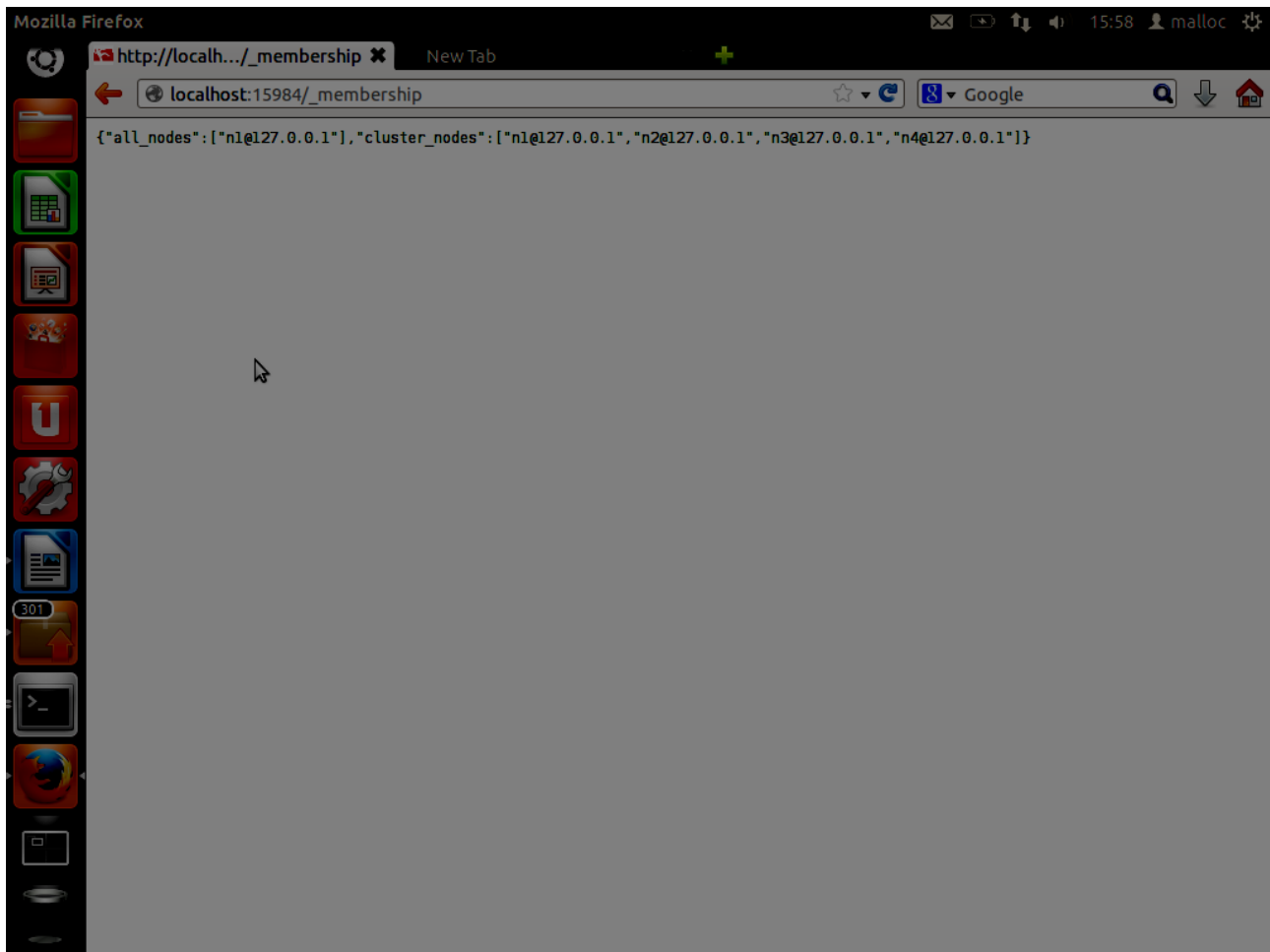
The bottom of the terminal shows the nano editor's command shortcuts: ^G Aide, ^O Écrire, ^R Lire fich., ^Y Page préc., ^K Couper, ^C Pos. cur., ^X Quitter, ^J Justifier, ^W Chercher, ^V Page suiv., ^U Coller, ^T Orthograp.

Une fois ce travail fait, notre architecture est mise en place, on peut maintenant lancer le(s) serveur(s) pour voir le résultat.

Pour lancer le nœud n1 on tape :

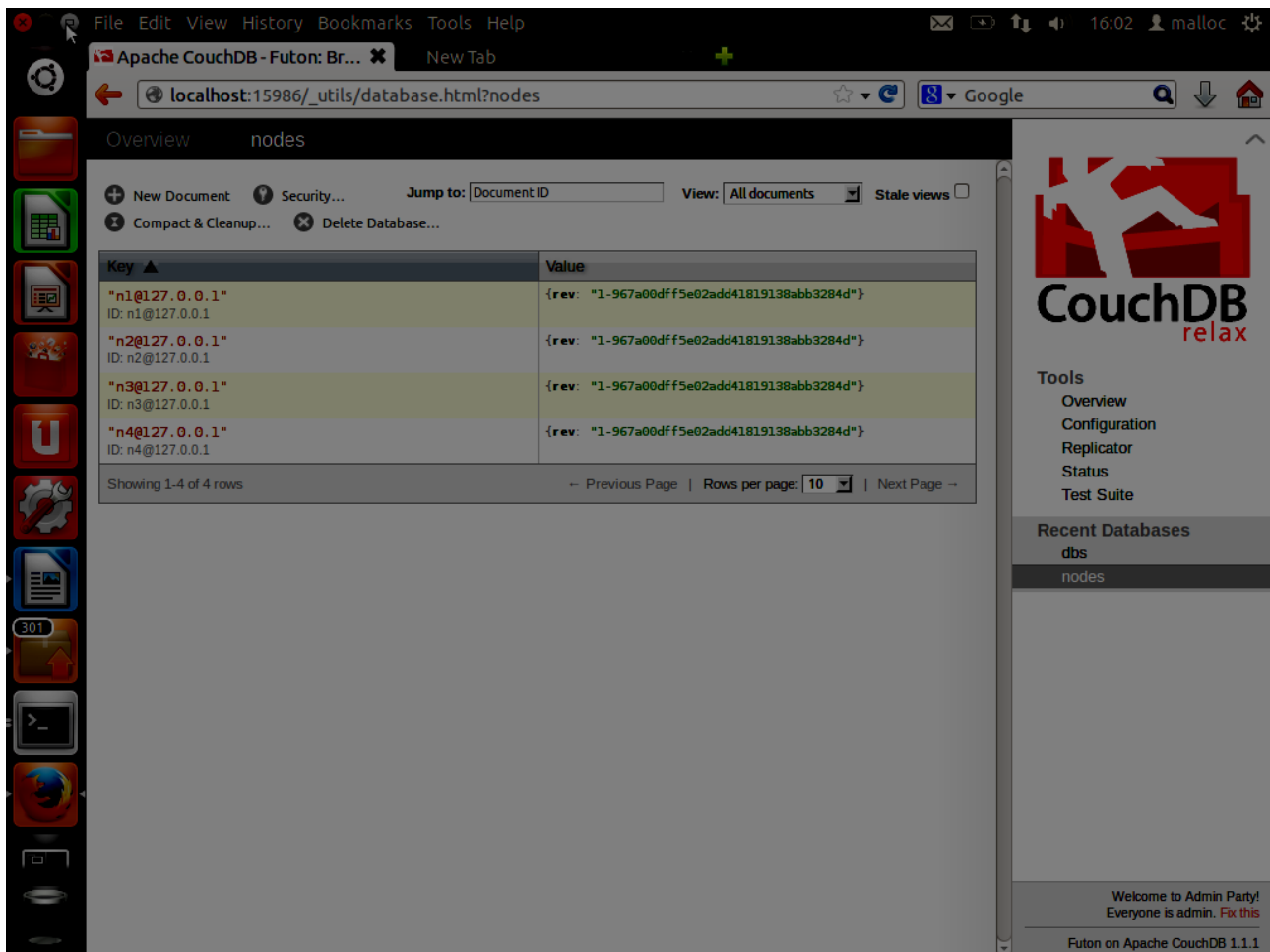
- bigcouch/rel/n1/bin/bigcouch, on peut aller voir ce qui passe sur les ports d'entrée 15984 et de sortie 15986

http://localhost:15984/_membership, (en entrée serveur n1) on a :



On remarque que ce serveur affiche la liste des quatre les nœuds ([n1@127.0.0.1](#), [n2@127.0.0.1](#), [n3@127.0.0.1](#), [n4@127.0.0.1](#)) il se comporte comme un routeur

http://localhost:15986/_utils/database.html?nodes) à la sortie on a:



Conclusion le serveur d'entrée, se comporte comme le maître de tous les nœuds en sortie, puisque sur ce port de sortie on aperçoit les tous serveurs (n1@127.0.0.1, n2@127.0.0.1, n3@127.0.0.1, et n4@127.0.0.1) ce serveur est le serveur BigCouch qui se rechargera de distribuer les données sur les fragments contenus dans les quatre nœuds. Nous avons donc besoin de lancer qu'un serveur avec son port d'entrée, automatiquement les autres se comporteront des esclaves pour recevoir des requêtes.

Dans ce qui nous allons maintenant voir comment s'effectue la répartition de charges, nous allons créer un espace de stockage dans le serveur maître ou routeur puis faire l'importation des 60,8 Mo.

En tapant `curl -X PUT http://localhost:15984/voteprojets` puis en vérifiant si la base a été créée à l'aide `curl -X GET http://localhost:15984/_all_dbs`, on obtient bien ["voteprojets"], la ressource voteprojets a été créée et elle est vide, nous allons maintenant faire l'importation (du fichier JSON de taille 60 Mo) en tapant depuis le terminal la commande :

```
curl -H "Content-type:application/json" -d @voteProjets.json -X POST  
http://127.0.0.1:15986/ voteprojets /_bulk_docs
```

Où faire de la réplication étant donné que voteprojets avait été importé déjà sur le serveur `http://127.0.0.1:5984/voteprojets` (dans la partie I)

Pour la réplication on tape :

```
curl -H "Content-type:application/json" -X POST
http://localhost:5984/_replicate -d
'{"source":"http://localhost:5984/voteprojets","target":"http://localhost:1598
4/voteprojets"}'
```

Nous avons malheureusement cette erreur insistante dans la suite des opérations :

```
<147>1 2015-02-28T20:11:04.676645Z ubuntu twig emulator msg - Error
in process <0.2250.0> on node 'n1@127.0.0.1' with exit value:
{badarg,[{erlang,binary_to_term,[<<0
bytes>>]}},{couch_file,pread_term,2},{couch_btree,get_node,2},{couch_bt
ree,stream_node,7},{couch_btree,fold,4},{couch_view,fold,4},{couch_db,e
num_docs,4},{mem3_shards,fold,2}]}
```

```
[error] [emulator] [-----] Error in process <0.2250.0> on node
'n1@127.0.0.1' with exit value: {badarg,[{erlang,binary_to_term,[<<0
bytes>>]}},{couch_file,pread_term,2},{couch_btree,get_node,2},{couch_bt
ree,stream_node,7},{couch_btree,fold,4},{couch_view,fold,4},{couch_db,e
num_docs,4},{mem3_shards,fold,2}]}
```

Qui nous empêche à repartir les données dans les fragments de chaque nœud

